HSB
Hochschule Bremen
City University of Applied Sciences

Project Proposal

Modeling Satellite Kinematics and Spatial Collision Detection via Numerical Integration

Prof. Dr. Martin Hering-Bertram

Richard Schröder: rschroeder@stud.hs-bremen.de
Khang Pham: kpham@stud.hs-bremen.de
Finn Rusche: frusche@stud.hs-bremen.de
Gian Isikli: gisikli@stud.hs-bremen.de

June 17, 2026

1 Abstract

This project investigates the mathematical modeling of satellite kinematics and the algorithmic detection of spatial collisions in Earth's orbit. By formulating orbital mechanics as a system of ordinary differential equations (ODEs) and solving them using numerical integration, the project simulates realistic trajectories. Concurrently, a spatial collision detection system utilizing bounding volumes is developed to predict and evaluate potential impact scenarios with asteroids or space debris.

2 Motivation, Application Scenario, Requirements

The increasing density of satellite constellations and space debris poses a severe threat to aerospace operations. In modern aerospace engineering and mission control—such as the operational environments characteristic of Bremen's aerospace sector—predicting trajectories and avoiding collisions is a critical requirement. The primary application scenario simulates a low Earth orbit satellite (LEO) navigating a debris field. The requirements for this project include accurate physical modeling of gravitational forces. Stable numerical integration to compute positions over time. Efficient algorithms to detect intersections between 3D objects without relying on computationally expensive triangle-to-triangle checks.

3 Related Work

The foundation of this project relies on classical orbital mechanics, specifically the numerical integration of Newton's law of universal gravitation. Previous works and standard libraries typically employ Runge-Kutta methods to solve the underlying initial value problems. For spatial intersections, concepts from computational geometry, specifically the use of boundary spheres and Axis-Aligned boundary boxes (AABB) as described in the real-time rendering literature, serve as the basis for the collision detection algorithms.

4 Development Target (Implementation and Examples)

The core development consists of two decoupled mathematical modules implemented within a structured technical architecture:

1. **Kinematics Engine:** The satellite's motion is governed by the ordinary differential equation:

$$m\frac{d\vec{v}}{dt} = \vec{F}_{grav} = -G\frac{M \cdot m}{\|\vec{r}\|^3}\vec{r} \tag{1}$$

where G is the gravitational constant, M is the Earth's mass, m is the satellite's mass, and $\vec{r}$ is the spatial position vector. This continuous system will be discretised and solved using adaptive numerical integration to output 3D coordinate arrays over a specified time vector.

2. **Collision Detection System:** We will implement an algorithm that wraps the target objects in mathematical spheres. A collision is defined and detected when the Euclidean distance between two coordinate centers is strictly less than the sum of their radii:

$$\|\vec{p}_1 - \vec{p}_2\| < r_1 + r_2 \tag{2}$$

5 Use of Tools

The implementation and documentation will strictly rely on professional engineering and development tools to ensure reproducible results:

- **Core Implementation:** Python 3 for high-level scripting and algorithm development.
- **Mathematical Operations:** `NumPy` for optimized vector/matrix operations and `SciPy` (specifically `scipy.integrate.solve_ivp` for ODE solving).
- **Visualization:** `Matplotlib` or `Plotly` for generating 3D plots of the calculated trajectories.
- **Version Control & Infrastructure:** `Git` for collaborative code management, versioning, and structural alignment.
- **Documentation:** LaTeX (via Overleaf) for compiling the technical layout.

6 Evaluation Methods and Expected Results

The evaluation framework will validate both mathematical precision and algorithmic efficiency:

- **Orbital Precision:** The numerical results of the two-body simulation will be compared against the analytical solutions derived from Kepler's laws. The cumulative error margin over multiple orbits will be quantified to evaluate long-term numerical stability.
- **Collision Accuracy:** The detection algorithm will be stress-tested with high-velocity intersection vectors to verify logical accuracy and timestamp precision.
- **Expected Outcomes:** A robust Python script capable of calculating critical collision windows and exporting structured 3D spatial plots for the subsequent technical report.

7 Work Schedule/Load

The project will be executed over a five-week timeline to meet all academic milestones. All team members will work on all aspects of this project:

Timeframe	**Milestones and Deliverables**
Week 1	Finalizing mathematical models (Gravitation and Bounding Volumes)
Week 2	Python implementation of `solve_ivp` and basic orbit plotting
Week 3	Implementation of collision algorithms and generating test scenarios
Week 4	Evaluation of numerical examples and drafting the LaTeX report
Week 5	Finalizing formatting, plotting graphs, and preparing the colloquium

References

[1] H. D. Curtis, *Orbital Mechanics for Engineering Students*, Butterworth-Heinemann, 2013.

[2] C. Ericson, *Real-Time Collision Detection*, CRC Press, 2005.

[3] SciPy Community, *SciPy Documentation: Integration and ODEs*, 2026.